

Neural Network Training Pipeline for Gravitational Waveform Emulation

Jim Emulators Project

December 2025

Abstract

This document details the implementation choices for training a neural network emulator of gravitational waveform modes. We analyse the characteristics of the training data, derive normalization strategies, describe the PCA compression scheme, and specify the network architecture. The primary target is the $(2,2)$ mode of IMRPhenomXHM for aligned-spin binary black holes.

Contents

1	Training Data Characteristics	3
1.1	Frequency Grid	3
1.2	Log-Amplitude Characteristics	4
1.3	Phase Characteristics	4
1.4	Parameter Space	4
2	Normalization Strategy	4
2.1	Output Normalization: Per-Frequency Standardization	5
2.2	Input Normalization: Parameter Standardization	6
3	PCA Compression	6
3.1	Separate PCA for Amplitude and Phase	7
3.2	Choosing the Number of Components	7
3.3	PCA Workflow	8
3.4	PCA Coefficient Standardization	8
4	Neural Network Architecture	9
4.1	Architecture Overview	9
4.2	Speculator Activation Function	9
4.3	Network Dimensions	10
4.4	Output Structure	10
5	Training Configuration	10
5.1	Loss Function	10
5.2	Optimizer	11
5.3	Training Schedule	11
5.4	Data Splits	11
6	Inference Pipeline	11
6.1	Forward Pass	12
6.2	Computational Cost	12
6.3	Full Pipeline Diagram	12

7	Validation Metrics	12
7.1	Training Metrics	12
7.2	Physics Metrics	13

1 Training Data Characteristics

Figure 1 shows the validation plots for 10,000 training waveforms generated on a log-spaced grid in geometric frequency $\mathcal{F} = Mf$.

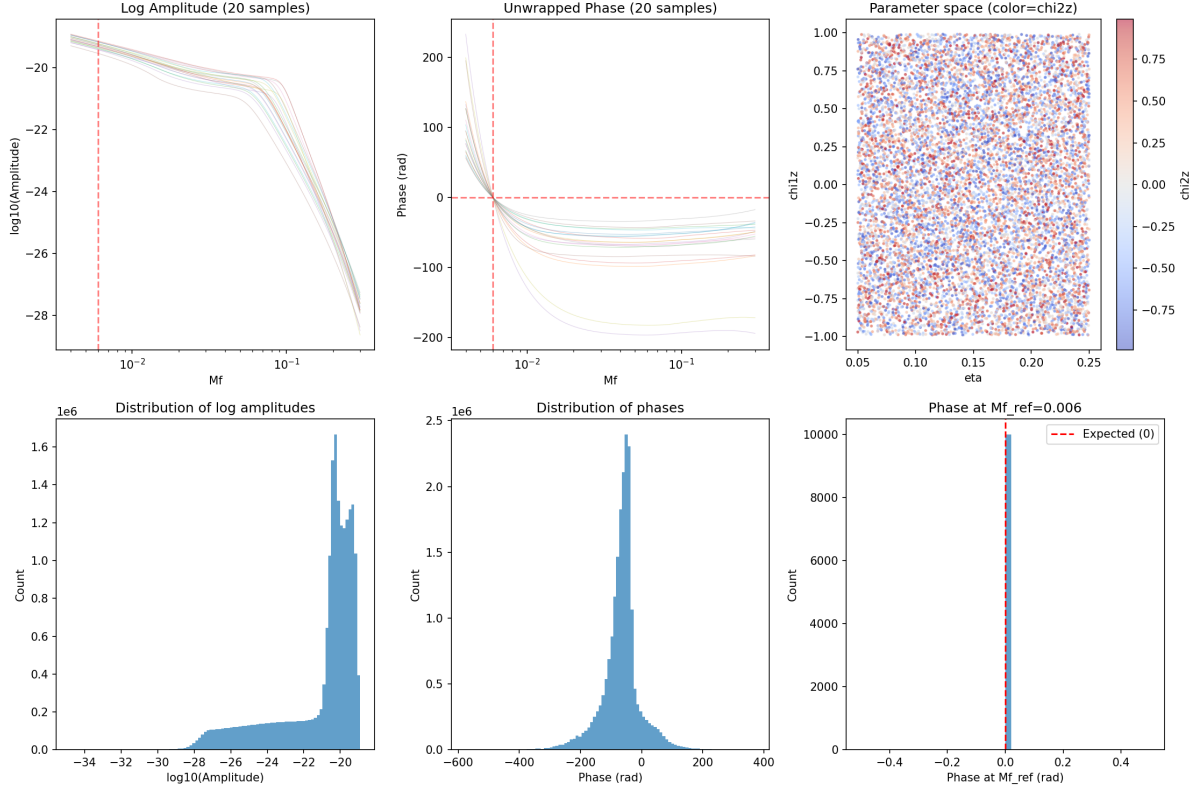


Figure 1: Validation plots for the (2,2) mode training data. **Top row:** Log-amplitude and unwrapped phase vs $\mathcal{F}$ for 20 representative samples, plus parameter space coverage. **Bottom row:** Marginal distributions of log-amplitude, phase, and phase at the reference frequency $\mathcal{F}_{\text{ref}} = 0.006$.

1.1 Frequency Grid

The training data is evaluated on a fixed log-spaced grid:

$$\mathcal{F}_k = \mathcal{F}_{\min} \times \left(\frac{\mathcal{F}_{\max}}{\mathcal{F}_{\min}} \right)^{k/(N-1)}, \quad k = 0, \dots, N-1 \quad (1)$$

with parameters:

Parameter	Value
$\mathcal{F}_{\min}$	0.004
$\mathcal{F}_{\max}$	0.3
N (grid points)	2000
$\mathcal{F}_{\text{ref}}$	0.006

The choice $\mathcal{F}_{\min} = 0.004$ avoids phase aliasing at low frequencies (where the phase evolves most rapidly). For a binary with total mass $M = 50 M_{\odot}$, this corresponds to $f_{\min} \approx 16$ Hz.

1.2 Log-Amplitude Characteristics

From Figure 1 (top-left and bottom-left):

- **Range:** $\log_{10} A \in [-26, -20]$ (span of ~ 6)
- **Shape:** Rises during inspiral, peaks near merger ($\mathcal{F} \approx 0.02$), decays in ringdown
- **Distribution:** Bimodal, reflecting the different frequency regimes (many grid points in the slowly-varying inspiral vs. fewer points near the sharp merger peak)
- **Variation across parameters:** Amplitude shape varies smoothly with $(\eta, \chi_{1z}, \chi_{2z})$

The bimodal distribution in the marginal histogram arises because:

1. The log-spaced grid places many points at low $\mathcal{F}$ (inspiral) where amplitudes are moderate
2. Fewer points sample the merger peak (high amplitude)
3. The ringdown tail contributes low-amplitude points at high $\mathcal{F}$

1.3 Phase Characteristics

From Figure 1 (top-middle and bottom-middle):

- **Range:** $\Phi \in [-200, 0]$ radians (span of ~ 200)
- **Shape:** Monotonically increasing with $\mathcal{F}$ (equivalently, monotonically decreasing backwards from the reference)
- **Alignment:** All waveforms satisfy $\Phi(\mathcal{F}_{\text{ref}}) = 0$ exactly (bottom-right panel)
- **Variation:** Phase accumulation varies significantly with parameters, particularly η (controls chirp rate)

The phase alignment at $\mathcal{F}_{\text{ref}}$ is critical: it removes a gauge freedom and ensures that all waveforms share a common reference point. The bottom-right histogram confirms this alignment is exact (delta function at zero).

1.4 Parameter Space

The training data covers the aligned-spin parameter space:

Parameter	Range	Physical meaning
η	$[0.05, 0.25]$	Symmetric mass ratio ($q \in [1, 18]$)
χ_{1z}	$[-0.99, 0.99]$	Primary spin (aligned)
χ_{2z}	$[-0.99, 0.99]$	Secondary spin (aligned)

Latin Hypercube Sampling ensures uniform coverage (Figure 1, top-right).

2 Normalization Strategy

Proper normalization is essential for stable training. We apply different normalization schemes to inputs (parameters) and outputs (waveforms).

2.1 Output Normalization: Per-Frequency Standardization

The key insight from Figure 1 is that both amplitude and phase vary *systematically* with frequency. A global normalization would be dominated by the extreme values, leaving most of the data poorly scaled.

We therefore apply **per-frequency standardization**: at each frequency bin k , we compute statistics across the training set and normalize independently.

Log-amplitude normalization. Let $\mathbf{A} \in \mathbb{R}^{N_{\text{train}} \times N_{\text{freq}}}$ be the matrix of log-amplitudes (each row is one waveform). Compute:

$$\mu_k^{(A)} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} A_{ik} \quad (2)$$

$$\sigma_k^{(A)} = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (A_{ik} - \mu_k^{(A)})^2} \quad (3)$$

To avoid numerical instability at frequencies where waveforms vary little across the parameter space, we apply a floor to the standard deviation:

$$\sigma_k^{(A)} \leftarrow \max(\sigma_k^{(A)}, \sigma_{\text{floor}}) \quad (4)$$

with $\sigma_{\text{floor}} = 10^{-6}$ (or a small fraction of the typical variation). This prevents division by near-zero values that would amplify numerical noise.

The normalized log-amplitude is:

$$\hat{A}_{ik} = \frac{A_{ik} - \mu_k^{(A)}}{\sigma_k^{(A)}} \quad (5)$$

Phase normalization. Similarly, let $\Phi \in \mathbb{R}^{N_{\text{train}} \times N_{\text{freq}}}$ be the phase matrix. Compute:

$$\mu_k^{(\Phi)} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \Phi_{ik} \quad (6)$$

$$\sigma_k^{(\Phi)} = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (\Phi_{ik} - \mu_k^{(\Phi)})^2} \quad (7)$$

The same σ_{floor} is applied to prevent instabilities. The normalized phase is:

$$\hat{\Phi}_{ik} = \frac{\Phi_{ik} - \mu_k^{(\Phi)}}{\sigma_k^{(\Phi)}} \quad (8)$$

Storage requirements. We must store the normalization constants for inference:

- $\mu^{(A)}, \sigma^{(A)} \in \mathbb{R}^{N_{\text{freq}}}$ for amplitude
- $\mu^{(\Phi)}, \sigma^{(\Phi)} \in \mathbb{R}^{N_{\text{freq}}}$ for phase

Total: $4 \times 2000 = 8000$ floats (~ 32 kB in float32).

Why per-frequency normalization?

1. **Balanced gradients:** Without it, the loss would be dominated by high- $|\Phi|$ points at low $\mathcal{F}$
2. **Equal PCA contribution:** All frequencies contribute equally to variance, ensuring PCA captures shape variation rather than magnitude
3. **Stable optimization:** Neural networks train best when inputs and outputs are $\mathcal{O}(1)$

Differentiability. A key requirement for use in gradient-based inference (e.g., Hamiltonian Monte Carlo in `jim`) is that the emulator must be differentiable with respect to the input parameters. Per-frequency normalization preserves differentiability because:

- The normalization constants $(\boldsymbol{\mu}, \boldsymbol{\sigma})$ are computed *once* from the training data and then *fixed*
- At inference, normalization and denormalization are simple linear transformations:

$$\hat{x}_k = \frac{x_k - \mu_k}{\sigma_k}, \quad x_k = \hat{x}_k \cdot \sigma_k + \mu_k \quad (9)$$

- These operations have trivial gradients: $\partial \hat{x}_k / \partial x_k = 1 / \sigma_k$ (constant)
- In JAX, the normalization constants are stored as static arrays, and all operations are compatible with `jax.grad` and `jax.jit`

The full pipeline (input normalization $\rightarrow$ MLP $\rightarrow$ PCA reconstruction $\rightarrow$ output denormalization) is end-to-end differentiable.

2.2 Input Normalization: Parameter Standardization

The input parameters have different ranges. We map all to $[-1, 1]$:

Symmetric mass ratio.

$$\hat{\eta} = \frac{2(\eta - 0.05)}{0.25 - 0.05} - 1 = 10\eta - 1.5 \quad (10)$$

Spin components. The spins are already nearly in $[-1, 1]$:

$$\hat{\chi}_{1z} = \frac{\chi_{1z}}{0.99}, \quad \hat{\chi}_{2z} = \frac{\chi_{2z}}{0.99} \quad (11)$$

Inverse transformations. For inference, we need the inverse mappings:

$$\eta = 0.1(\hat{\eta} + 1.5) = 0.1\hat{\eta} + 0.15 \quad (12)$$

$$\chi_{1z} = 0.99 \hat{\chi}_{1z}, \quad \chi_{2z} = 0.99 \hat{\chi}_{2z} \quad (13)$$

3 PCA Compression

After normalization, we apply Principal Component Analysis to compress the waveforms from $N_{\text{freq}} = 2000$ dimensions to $K \ll N_{\text{freq}}$ components.

Important: No double standardization. The input to PCA is the *already-normalized* data $\hat{\mathbf{A}}$ and $\hat{\Phi}$ from §2. The PCA step should **not** re-standardize the data internally. Since the per-frequency normalization produces zero-mean data at each frequency, PCA centering is already satisfied. Applying a second standardization would conflict with the per-frequency normalization and produce incorrect coefficients.

3.1 Separate PCA for Amplitude and Phase

We perform PCA separately on the normalized amplitude and phase matrices:

$$\hat{\mathbf{A}} = \mathbf{C}^{(A)} \mathbf{V}^{(A)\top} + \text{truncation error} \quad (14)$$

$$\hat{\Phi} = \mathbf{C}^{(\Phi)} \mathbf{V}^{(\Phi)\top} + \text{truncation error} \quad (15)$$

where:

- $\mathbf{V}^{(A)} \in \mathbb{R}^{K_A \times N_{\text{freq}}}$ are the amplitude principal components
- $\mathbf{V}^{(\Phi)} \in \mathbb{R}^{K_\Phi \times N_{\text{freq}}}$ are the phase principal components
- $\mathbf{C}^{(A)} \in \mathbb{R}^{N_{\text{train}} \times K_A}$ are the amplitude coefficients
- $\mathbf{C}^{(\Phi)} \in \mathbb{R}^{N_{\text{train}} \times K_\Phi}$ are the phase coefficients

Why separate PCA? Amplitude and phase have fundamentally different characteristics:

- Amplitude is positive, has a peak structure, decays at edges
- Phase is monotonic, accumulates hundreds of radians

Mixing them in a joint PCA would require the basis to simultaneously capture both behaviours, likely requiring more components.

3.2 Choosing the Number of Components

We select K_A and K_Φ to retain a target fraction of the variance:

$$\frac{\sum_{k=1}^K \lambda_k}{\sum_{k=1}^{N_{\text{freq}}} \lambda_k} \geq 1 - \epsilon \quad (16)$$

where λ_k are the eigenvalues (sorted descending) and ϵ is the truncation tolerance.

Recommended tolerance. For waveform emulation targeting mismatch $\mathcal{M} < 10^{-3}$, we require:

$$\epsilon = 10^{-4} \quad (\text{retain 99.99\% variance}) \quad (17)$$

This is conservative; the PCA truncation error should be subdominant to the neural network approximation error.

Expected number of components. Based on the smoothness of gravitational waveforms and results from similar emulators (CosmoPower, mlgw_NN), we expect:

- $K_A \sim 30\text{--}80$ for amplitude
- $K_\Phi \sim 30\text{--}80$ for phase

The exact values will be determined empirically from the training data.

3.3 PCA Workflow

Algorithm 1 PCA Compression (Training)

Require: Normalized data $\hat{\mathbf{A}}, \hat{\Phi} \in \mathbb{R}^{N_{\text{train}} \times N_{\text{freq}}}$

Require: Variance threshold $1 - \epsilon$

Ensure: Principal components $\mathbf{V}^{(A)}, \mathbf{V}^{(\Phi)}$

Ensure: Coefficients $\mathbf{C}^{(A)}, \mathbf{C}^{(\Phi)}$

- 1: Compute SVD: $\hat{\mathbf{A}} = \mathbf{U}^{(A)} \mathbf{S}^{(A)} \mathbf{V}^{(A)\top}$
 - 2: Select K_A such that $\sum_{k=1}^{K_A} s_k^2 / \sum_k s_k^2 \geq 1 - \epsilon$
 - 3: Truncate: $\mathbf{V}^{(A)} \leftarrow \mathbf{V}_{:,K_A,:}^{(A)}$
 - 4: Coefficients: $\mathbf{C}^{(A)} \leftarrow \hat{\mathbf{A}} \mathbf{V}^{(A)\top}$
 - 5: Repeat for phase: $\hat{\Phi} \rightarrow \mathbf{V}^{(\Phi)}, \mathbf{C}^{(\Phi)}$
-

Algorithm 2 PCA Reconstruction (Inference)

Require: PCA coefficients $\mathbf{c}^{(A)} \in \mathbb{R}^{K_A}, \mathbf{c}^{(\Phi)} \in \mathbb{R}^{K_\Phi}$

Require: Principal components $\mathbf{V}^{(A)}, \mathbf{V}^{(\Phi)}$

Ensure: Normalized waveform $\hat{\mathbf{a}}, \hat{\phi} \in \mathbb{R}^{N_{\text{freq}}}$

- 1: $\hat{\mathbf{a}} \leftarrow \mathbf{c}^{(A)} \mathbf{V}^{(A)}$ $\triangleright$ Matrix-vector product
 - 2: $\hat{\phi} \leftarrow \mathbf{c}^{(\Phi)} \mathbf{V}^{(\Phi)}$
-

3.4 PCA Coefficient Standardization

Following CosmoPower [2], the PCA coefficients are standardized before being used as training targets. This ensures all coefficients have comparable scale, which improves neural network training stability.

For each PCA coefficient j , compute the mean and standard deviation across the training set:

$$\mu_j^{(c)} = \frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} C_{ij} \quad (18)$$

$$\sigma_j^{(c)} = \sqrt{\frac{1}{N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} (C_{ij} - \mu_j^{(c)})^2} \quad (19)$$

The standardized coefficients are:

$$\hat{C}_{ij} = \frac{C_{ij} - \mu_j^{(c)}}{\sigma_j^{(c)}} \quad (20)$$

This standardization is applied separately to the amplitude and phase coefficients. At inference, the network outputs standardized coefficients, which are then denormalized:

$$C_{ij} = \hat{C}_{ij} \cdot \sigma_j^{(c)} + \mu_j^{(c)} \quad (21)$$

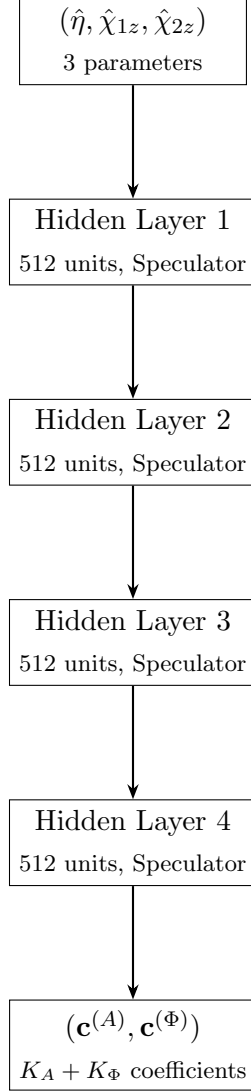
Storage requirements. We must store:

- $\mu^{(c,A)}, \sigma^{(c,A)} \in \mathbb{R}^{K_A}$ for amplitude coefficients
- $\mu^{(c,\Phi)}, \sigma^{(c,\Phi)} \in \mathbb{R}^{K_\Phi}$ for phase coefficients

4 Neural Network Architecture

Following CosmoPower and Speculator, we use a Multi-Layer Perceptron (MLP) to map parameters to PCA coefficients.

4.1 Architecture Overview



4.2 Speculator Activation Function

The Speculator activation [1] is a learnable gated linear unit:

$$\sigma(x) = x \cdot \text{sigmoid}(\alpha x) \cdot (1 - \gamma) + \gamma \cdot x \quad (22)$$

where $\gamma \in (0, 1)$ controls the linear bypass and α is a learnable scale. In practice, we parameterize γ via a sigmoid:

$$\gamma = \text{sigmoid}(\gamma_{\text{logit}}) \quad (23)$$

with γ_{logit} initialized to -2.0 (mostly gated, empirically optimal).

Why Speculator activation?

- Smooth, differentiable everywhere

- Learnable nonlinearity adapts to the problem
- Proven effective for smooth function emulation
- Avoids dying ReLU problem

4.3 Network Dimensions

Layer	Input dim	Output dim
Input	—	3
Hidden 1	3	512
Hidden 2	512	512
Hidden 3	512	512
Hidden 4	512	512
Output	512	$K_A + K_\Phi$

Total parameters. Assuming $K_A = K_\Phi = 64$ (128 total output):

$$N_{\text{params}} = (3 \times 512 + 512) + 3 \times (512 \times 512 + 512) + (512 \times 128 + 128) \quad (24)$$

$$= 2048 + 788480 \times 3 + 65664 \quad (25)$$

$$\approx 854,000 \quad (26)$$

Plus learnable activation parameters (small overhead).

4.4 Output Structure

The network outputs a single vector of dimension $K_A + K_\Phi$:

$$\mathbf{y} = \text{MLP}(\hat{\eta}, \hat{\chi}_{1z}, \hat{\chi}_{2z}) = [\mathbf{c}^{(A)}, \mathbf{c}^{(\Phi)}] \in \mathbb{R}^{K_A + K_\Phi} \quad (27)$$

Alternative: separate networks. One could train separate networks for amplitude and phase coefficients. This would allow different architectures but increases complexity. We start with a joint network.

5 Training Configuration

5.1 Loss Function

We minimize the mean squared error on PCA coefficients:

$$\mathcal{L} = \frac{1}{N_{\text{batch}}} \sum_{i=1}^{N_{\text{batch}}} \left[\left\| \mathbf{c}_i^{(A)} - \hat{\mathbf{c}}_i^{(A)} \right\|^2 + \left\| \mathbf{c}_i^{(\Phi)} - \hat{\mathbf{c}}_i^{(\Phi)} \right\|^2 \right] \quad (28)$$

where $\hat{\mathbf{c}}$ denotes the network prediction.

Why MSE on coefficients?

- Simple, well-understood
- PCA coefficients are already normalized (zero mean, unit variance for the first components)
- Directly minimizes reconstruction error in the PCA basis

Alternative: reconstruction loss. One could compute MSE on the reconstructed waveform:

$$\mathcal{L}_{\text{recon}} = \left\| \hat{\mathbf{A}}_{\text{true}} - \hat{\mathbf{A}}_{\text{pred}} \right\|^2 + \left\| \hat{\Phi}_{\text{true}} - \hat{\Phi}_{\text{pred}} \right\|^2 \quad (29)$$

This is mathematically equivalent (up to a constant factor) when using the full PCA basis, but differs when truncating. The coefficient loss is more efficient (no reconstruction needed during training).

5.2 Optimizer

We use Adam with gradient clipping:

Hyperparameter	Value
Optimizer	Adam
Learning rate	10^{-3} (initial)
Learning rate schedule	Cosine decay to 10^{-5}
Gradient clipping	Global norm ≤ 1.0
Weight decay	10^{-5} (optional)

5.3 Training Schedule

Parameter	Value
Batch size	256
Epochs	1000 (with early stopping)
Validation frequency	Every epoch
Early stopping patience	50 epochs

5.4 Data Splits

From the generated dataset:

- Training: 10,000 samples (90%)
- Validation: 1,000 samples (10%)

The validation set uses a different random seed, ensuring no overlap with training.

6 Inference Pipeline

At inference time, we reconstruct waveforms from physical parameters via the trained emulator.

6.1 Forward Pass

Algorithm 3 Waveform Emulation (Inference)

Require: Physical parameters $(\eta, \chi_{1z}, \chi_{2z})$

Require: Trained MLP, PCA components, normalization constants

Ensure: Log-amplitude $\mathbf{A}$ and phase Φ on $\mathcal{F}$ grid

```

1: Normalize inputs:
2:  $\hat{\eta} \leftarrow 10\eta - 1.5$ 
3:  $\hat{\chi}_{1z} \leftarrow \chi_{1z}/0.99, \quad \hat{\chi}_{2z} \leftarrow \chi_{2z}/0.99$ 
4: Neural network forward pass:
5:  $[\hat{\mathbf{c}}^{(A)}, \hat{\mathbf{c}}^{(\Phi)}] \leftarrow \text{MLP}(\hat{\eta}, \hat{\chi}_{1z}, \hat{\chi}_{2z})$  ▷ Standardized coefficients
6: Denormalize PCA coefficients:
7:  $c_j^{(A)} \leftarrow \hat{c}_j^{(A)} \cdot \sigma_j^{(c,A)} + \mu_j^{(c,A)}$ 
8:  $c_j^{(\Phi)} \leftarrow \hat{c}_j^{(\Phi)} \cdot \sigma_j^{(c,\Phi)} + \mu_j^{(c,\Phi)}$ 
9: PCA reconstruction:
10:  $\hat{\mathbf{a}} \leftarrow \mathbf{c}^{(A)} \mathbf{V}^{(A)}$  ▷  $K_A \times N_{\text{freq}}$  matrix-vector
11:  $\hat{\phi} \leftarrow \mathbf{c}^{(\Phi)} \mathbf{V}^{(\Phi)}$ 
12: Denormalize outputs:
13:  $A_k \leftarrow \hat{a}_k \cdot \sigma_k^{(A)} + \mu_k^{(A)}$  ▷ Per-frequency
14:  $\Phi_k \leftarrow \hat{\phi}_k \cdot \sigma_k^{(\Phi)} + \mu_k^{(\Phi)}$ 
15: return  $\mathbf{A}, \Phi$ 

```

6.2 Computational Cost

Per waveform evaluation:

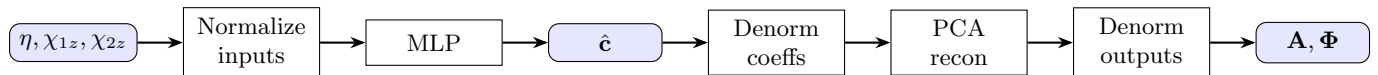
- Input normalization: 3 multiplications, 3 additions
- MLP forward pass: $\sim 850,000$ FLOPs (dominated by matrix multiplications)
- PCA reconstruction: $2 \times K \times N_{\text{freq}} \approx 256,000$ FLOPs
- Output denormalization: $2 \times N_{\text{freq}}$ multiplications and additions

Total: ~ 1.1 million FLOPs per waveform. On modern hardware:

- CPU: ~ 0.1 – 1 ms per waveform
- GPU (batched): < 0.01 ms per waveform

This represents a $\sim 1000\times$ speedup over LALSimulation.

6.3 Full Pipeline Diagram



7 Validation Metrics

7.1 Training Metrics

During training, we monitor:

- Training loss (MSE on PCA coefficients)

- Validation loss
- Per-component MSE (to identify problematic modes)

7.2 Physics Metrics

After training, we evaluate:

Reconstruction error. On the normalized waveform:

$$\epsilon_{\text{recon}} = \frac{1}{N_{\text{test}}} \sum_i \left(\left\| \hat{\mathbf{A}}_i - \hat{\mathbf{A}}_i^{\text{pred}} \right\|_{\infty} + \left\| \hat{\Phi}_i - \hat{\Phi}_i^{\text{pred}} \right\|_{\infty} \right) \quad (30)$$

Mismatch. The noise-weighted overlap (see theory document §6.1):

$$\mathcal{M} = 1 - \max_{t_c, \phi_c} \frac{\langle h_{\text{true}} | h_{\text{pred}} \rangle}{\sqrt{\langle h_{\text{true}} | h_{\text{true}} \rangle \langle h_{\text{pred}} | h_{\text{pred}} \rangle}} \quad (31)$$

Target: $\mathcal{M} < 10^{-3}$ for parameter estimation quality.

Figure 2 shows a representative comparison between the trained emulator and LAL ground truth, demonstrating excellent agreement in both frequency and time domains with mismatch $\mathcal{M} \approx 2.5 \times 10^{-4}$.

Relative amplitude error.

$$\epsilon_A = \max_k \left| \frac{A_k^{\text{pred}} - A_k^{\text{true}}}{A_k^{\text{true}}} \right| \quad (32)$$

Target: $< 1\%$

Phase error.

$$\epsilon_{\Phi} = \max_k \left| \Phi_k^{\text{pred}} - \Phi_k^{\text{true}} \right| \quad (33)$$

Target: < 0.1 radians

References

References

- [1] J. Alsing, B. Wandelt, S. Feeney, “Speculator: Emulating stellar population synthesis for fast and accurate galaxy spectra and photometry,” *Astrophys. J. Suppl.* **249**, 5 (2020), arXiv:1911.11778.
- [2] A. Spurio Mancini et al., “CosmoPower: emulating cosmological power spectra for accelerated Bayesian inference,” *Mon. Not. Roy. Astron. Soc.* **511**, 1771 (2022), arXiv:2106.03846.
- [3] S. Schmidt et al., “Machine learning gravitational waveforms for binary black hole mergers,” arXiv:2402.06587 (2024).

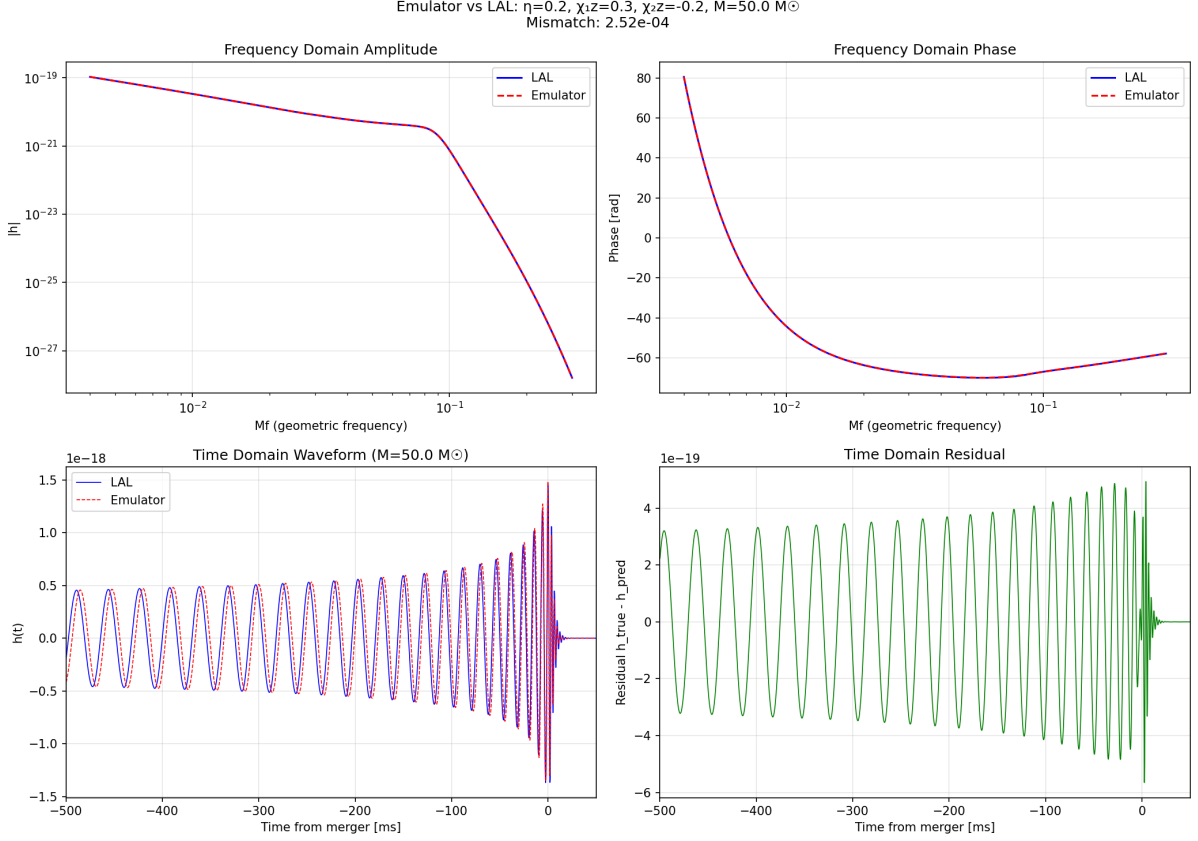


Figure 2: Emulator validation against LAL for a representative test case ($\eta = 0.2$, $\chi_{1z} = 0.3$, $\chi_{2z} = -0.2$, $M = 50 M_\odot$). **Top row:** Frequency domain amplitude and phase showing excellent agreement between LAL (blue) and emulator (red). **Bottom left:** Time domain waveform showing the inspiral chirp approaching merger at $t = 0$, with frequency increasing as expected for a gravitational wave. **Bottom right:** Time domain residual. The weighted mismatch is 2.52×10^{-4} , well below our target of 10^{-3} .